EnerAgent: Energy-Aware Code Refinement with LLM-Based Agents

Jonas Scheffler [1] and Max Westing [1]

Abstract: Large Language Models (LLMs) have shown strong performance in generating functionally correct source code, but their outputs often lack consideration for runtime efficiency and sustainability. In this work, we introduce EnerAgent, an agentic framework that uses iterative feedback to refine LLM-generated code toward reduced energy consumption. EnerAgent first ensures functional correctness, then performs iterative optimization guided by energy usage measurements. We evaluate the agent on a set of complex algorithmic problems in C++, comparing against both single-attempt LLM outputs and expert-written solutions. Our results show that EnerAgent consistently improves the energy efficiency of its initial code, with energy savings in over half of the tasks, often matching or surpassing human-written code. These findings highlight the promise of feedback-driven LLM agents for greener software development.

Keywords: Energy-Efficient Software, Large Language Models, Code Optimization, Autonomous Agents, Green Software Engineering

1 Introduction

Large Language Models (LLMs) have demonstrated impressive capabilities in code generation, achieving near-human performance across a range of programming tasks [Ji24]. This progress has sparked interest in using LLMs not just for functional correctness, but also for optimizing non-functional properties such as runtime, memory usage, and energy consumption.

Energy-efficient programming remains challenging as it requires deep knowledge of algorithmic trade-offs, hardware behavior, and the specific problem domain. For LLMs, the challenge is greater: efficiency is not reflected in their training objectives, and energy-aware behavior must emerge implicitly [SWC25]. To address this, recent works have explored agentic approaches where LLMs iteratively refine their outputs based on runtime feedback [Hu24; Ya23].

In this work, we introduce *EnerAgent*, an energy-aware agent that combines LLM-based code generation with feedback-driven refinement. The agent generates candidate solutions,

[1] Environmental Campus Birkenfeld, Trier University of Applied Sciences, Germany,
jonas.scheffler1@gmail.de, https://orcid.org/0009-0001-8606-8916;
m.westing@umwelt-campus.de, https://orcid.org/0009-0001-5044-7578
This paper was written as part of the course “Nachhaltige Softwaretechnik” taught by Prof. Dr. Stefan Naumann.

doi:10.18420/env2025_09

executes them to obtain runtime feedback, and iteratively improves the code by addressing inefficiencies. Unlike static, single-attempt code generation, this loop enables targeted exploration of the solution space to improve energy performance.

We evaluate EnerAgent on a suite of C++ programming tasks, comparing its performance to both human-written solutions and unrefined LLM outputs. EnerAgent often matches or surpasses human baselines in energy use, highlighting the potential of agentic LLMs for sustainable code generation.[2]

2 Related Works

As LLMs gain traction in software development, interest is growing in using them not just for functional correctness but also for efficient, sustainable code generation. Future software engineering will likely integrate LLMs with energy-aware capabilities, both in code output and in model operation [SYL24].

Initial work has focused on prompt design to steer models toward energy-efficient solutions. Minor changes in phrasing or explicit instructions can impact energy characteristics, though such gains are often inconsistent and task-specific [CRO24; Ni24].

More robust are agentic approaches, where LLMs refine outputs based on feedback from real execution [Ya23]. Huang et al. [Hu24] proposed *EffiLearner*, an iterative system for optimizing Python code using runtime and memory feedback, showing the promise of performance-aware refinement.

Our work builds on this paradigm with a fully automated design focused on energy as the primary metric, and aims to generalize across languages via Dockerized execution and energy tracking.

3 Methodology

3.1 EnerAgent Framework

In this work, we define **energy** as the amount of computational energy consumed by a program during its execution, measured using the CodeCarbon framework [CSL+24]. A solution is considered **energy-efficient** if it minimizes energy consumption while preserving functional correctness.

EnerAgent is a multi-stage, feedback-driven agent built to generate energy-efficient solutions to algorithmic programming tasks. Implemented using LangGraph [CC25], it structures the

[2] All source code and data for reproducing our results are available at: https://gitlab.rlp.net/s20a8d902543/eneragent-public

generation and refinement process as a directed state machine, where each node represents a distinct phase in the workflow.

The agent proceeds through the following structured phases:

- **Initial Code Generation:** An initial solution is generated via a zero-shot prompt [Ma20] that instructs the LLM to solve the task in C++ using the provided problem description and insert the solution into the given function template.
- **Compilation and Testing:** The generated code is compiled and executed inside a Docker-based execution environment. Code correctness is verified against predefined functional tests. If compilation or execution fails, the agent triggers a reflection step.
- **Reflection and Repair:** If the code fails to compile or pass the functional tests, the agent prompts the LLM with the problem description, current code, and error messages, explicitly instructing it to fix the issue based on the feedback. This process iterates until a working solution is found or a maximum number of repair attempts is reached.
- **Energy Optimization:** Once functional correctness is achieved, the agent transitions into the optimization phase, where it iteratively prompts the LLM to improve the code's energy efficiency. The prompt follows a chain-of-thought structure [We22], asking the model to analyze the code step by step, identify inefficiencies related to control flow, data structures, or memory use, and then propose an improved version. If no inefficiencies are found, the model is instructed to respond with `NO_CHANGE` to terminate the optimization phase, preventing unnecessary computation and overfitting to noise in energy measurements. Energy consumption is measured during each execution using the CodeCarbon framework [CSL+24]. The agent keeps track of the best-performing solution and continues the loop as long as improvements are detected. If a newly proposed solution is functionally incorrect, it is discarded, and the agent resumes optimization from the last verified correct version. This prevents the agent from becoming stuck attempting to fix broken variants that may not yield energy gains. This energy feedback mechanism is designed to be fast and lightweight, allowing the agent to iteratively optimize solutions without introducing significant overhead. More complex or hardware-based measurement techniques (e.g., external power meters) would make real-time agent optimization impractically slow. Our approach prioritizes a low-latency, transferable energy signal that can run fully autonomously, making the agent applicable in real-world settings where runtime constraints matter.

The exact prompt templates used during code generation, error repair, and energy optimization are listed in the appendix.

To encourage exploration, EnerAgent increases the LLM's temperature after each failed optimization attempt—starting with low-temperature outputs to address obvious inefficiencies, then gradually generating more diverse variants. While this strategy is intuitively motivated

and supported by prior work on temperature effects in LLMs [Pe24; Re24], we did not isolate its contribution in our benchmark.

Fig. 1 illustrates EnerAgent's workflow as a directed graph of states, supporting modular handling of code generation, verification, and energy-aware refinement.

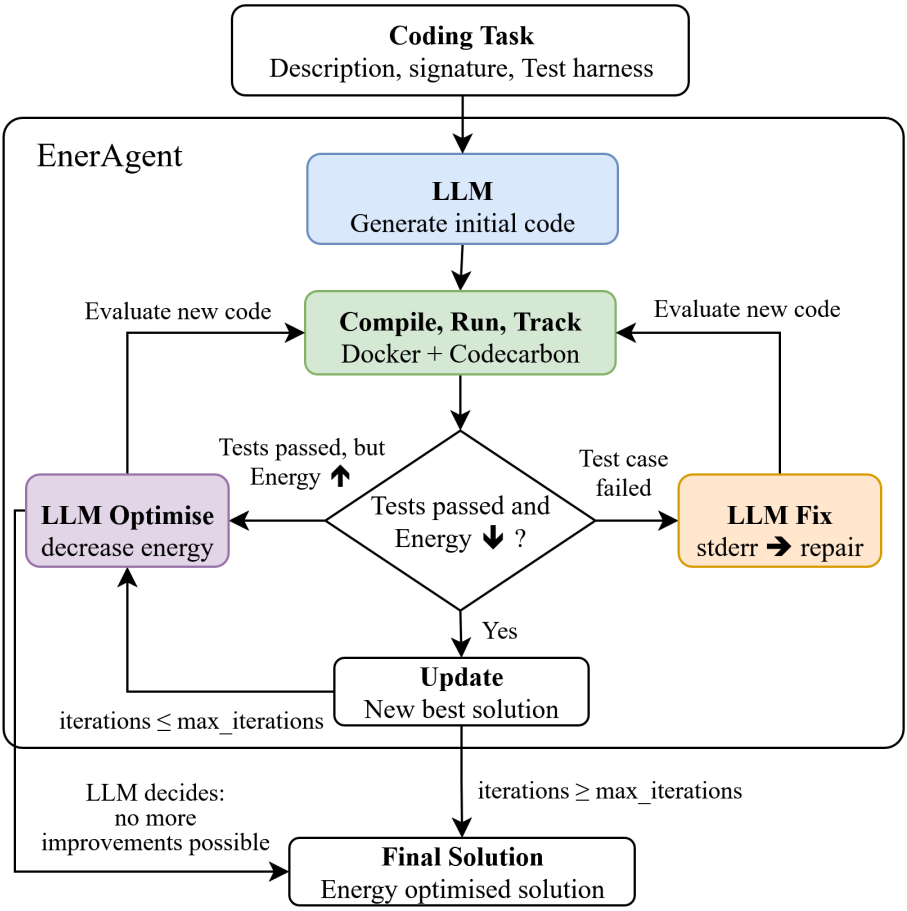

Fig. 1: **Eneragent Workflow.**The agent generates an initial solution using an LLM and a code template, then compiles, tests, and measures energy consumption. If incorrect, it enters a repair loop. Once correct, it performs iterative optimization to reduce energy use. The process ends after a fixed number of non-improving steps or when refinement is halted.

3.2 Experiment Design

To evaluate EnerAgent, we conducted experiments on a benchmark of algorithmic programming tasks drawn from LeetCode [3], focusing on problems labeled as "hard" to ensure sufficient complexity and challenge. These tasks typically require non-trivial algorithmic reasoning, such as dynamic programming, graph traversal, backtracking, or advanced data structure manipulation. Each task includes a natural language problem description, a C++ function signature template that defines the expected interface, and a set of functional tests used to verify correctness of candidate solutions. We chose C++ because its low-level nature allows for fine-grained optimizations, making it a more suitable language for evaluating energy-aware code refinement.

We compare the following methods:

- **Human Baseline:** Solutions manually written by expert programmers, measured for energy consumption.
- **LLM Single-attempt:** Code generated by prompting the LLM once, without refinement or feedback.
- **EnerAgent:** Our agentic framework that iteratively refines its output using feedback. It may attempt up to three repairs for incorrect code and continues energy optimization until either three consecutive non-improving attempts occur, or the LLM explicitly halts the loop.

Energy measurements were collected using CodeCarbon within Dockerized execution environments.

4 Results

We evaluate the methods along two primary axes: **effectiveness**, measured by the success rate of generating functionally correct code and the number of repair attempts required; and **efficiency**, assessed by the energy consumption of the generated code compared to human-written solutions and the degree of improvement achieved through iterative optimization.

To standardize our comparison of energy efficiency, we define the **Relative Energy Speedup (RES)** as the proportion of energy saved compared to a given baseline:

$$\text{RES} = \frac{E_{\text{baseline}} - E_{\text{agent}}}{E_{\text{baseline}}} \tag{1}$$

This metric reflects the relative reduction in energy consumption achieved by the agent. A higher RES indicates better energy efficiency. We apply this metric both in relation to the agent's initial solution (Sect. 4.2) and to human-written code (Sect. 4.3).

[3] https://leetcode.com

4.1 Functional Correctness (Effectiveness)

Tab. 1 summarizes EnerAgent's effectiveness in producing functionally correct solutions. Overall, the agent achieves a high success rate of over 80% for two different LLMs. A substantial portion of problems (over 60% for both LLMs) are solved in a single generation without any need for repair.

This high single-attempt success rate is unsurprising given that modern LLMs are extensively trained on code datasets, particularly on well-structured domains like competitive programming problems. However, initial generations can still suffer from minor issues, such as syntax errors, or failure to handle edge cases, that prevent compilation or correct execution.

A simple retry in which the error message is fed back into the LLM addresses many of these minor failures effectively. Almost all solved errors were corrected after just one additional refinement. Notably, very few tasks could be repaired with more than a single retry, which suggests that while a simple retry effectively corrects minor implementation errors, persistent failures are likely due to fundamental misunderstandings that cannot be resolved through additional attempts. Consequently, for tasks where functional correctness is achieved, the focus naturally shifts to optimizing other quality dimensions, such as energy efficiency, which we investigate in the following section.

Model	**Tasks**	**Solved (%)**	**1-attempt (%)**	**Fix attempts**		
				0	1	2
GPT-4.1	44	88.6	70.5	31	7	1
GPT-4.1-nano	44	81.8	61.4	27	8	1

Tab. 1: Summary of task outcomes: total attempted, percentage solved, single-attempt rate, and repair attempts required (0–2).

4.2 Energy Consumption (Efficiency)

One of the core capabilities of EnerAgent lies in its ability to iteratively improve code with respect to energy efficiency. As shown in Tab. 2, both model variants successfully reduced the energy consumption of their initial functional solutions in a significant portion of tasks. This demonstrates that even when an LLM can already produce functionally correct code, there is often considerable room for optimization, which may initially be discarded by prioritizing a simpler, but less efficient solution. The fact that over half of the tasks benefited from optimization reinforces the importance of a refinement stage for improving the quality of generated code.

The histogram in Sect. 4.2 visualizes the distribution of energy improvements across optimized tasks. Most improvements fall within the 10–20% range. While some degree of

Model	Successful opt.	Mean opt. attempts	Mean RES vs. Initial (%)
GPT-4.1	28	4.86	23.25
GPT-4.1-nano	18	3.17	18.54

Tab. 2: Optimization outcomes: success count, attempts, and RES vs. Initial code.

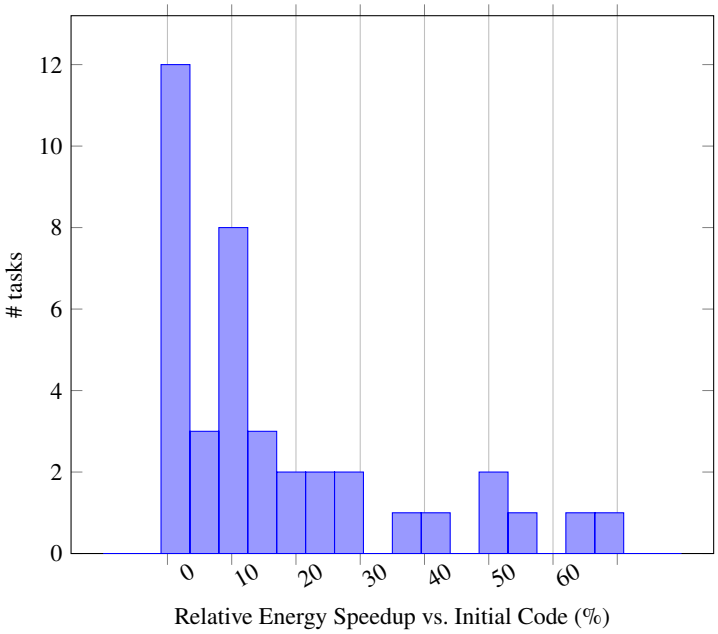

Fig. 2: Distribution of Relative Energy Speedup (RES) between initial and final agent solutions. Only functionally correct solutions are included.

the measured improvements may be explained by observational error, manual inspection of selected solutions indicated that they often stemmed from low-level optimizations tailored to the specific test conditions. These changes did not affect algorithmic complexity but instead reduced runtime overhead, such as avoiding unnecessary memory allocations or unrolling short loops. This can yield measurable speedups on small input sizes. However, these optimizations may not generalize and could even degrade performance on larger inputs due to reduced scalability.

More substantial improvements exceeding 30% were also observed, with some tasks showing energy reductions greater than 60%. These larger gains typically stem from the correction of more severe inefficiencies in the initial solution. In such cases, the model often replaced suboptimal algorithms or inappropriate data structures with more efficient alternatives, leading to the best possible runtime and memory complexity to solve the problem with.

These examples underscore the agent's ability to not just polish code, but to meaningfully rework it for greater efficiency when needed.

We emphasize that reported energy savings reflect relative differences under controlled conditions and are intended primarily as a feedback signal for guiding agent behavior, as explained in Sect. 3.1. These measurements may not be accurate estimates of absolute energy use. A more precise evaluation of true energy savings would require re-running optimized code independently, over extended durations, using external measurement tools—an avenue we leave to future work.[4]

4.3 Energy Efficiency Relative to Human Solutions

To assess the energy efficiency of the agent-generated code in a broader context, we compare its performance against human-written reference solutions. Tab. 3 shows the percentage of energy consumed by the final agent-generated code relative to the human implementation, considering only those tasks where the agent produced a functionally correct solution.

Model	Tasks	Mean RES vs. Human (%)	Median RES vs. Human (%)
GPT-4.1	39	16.53	10.00
GPT-4.1-nano	36	12.88	8.32

Tab. 3: Relative Energy Speedup (RES) of final agent solutions compared to human-written code.

These results suggest that the code generated through multiple rounds of refinement is competitive with human implementations in terms of energy consumption. However, they must be interpreted with nuance. In most cases, the final agent solutions converged to implementations of similar algorithmic complexity as their human-written counterparts. The observed energy savings primarily stemmed from small optimizations that exploited the limited input sizes used during testing (as discussed in Sect. 4.2).

In a few tasks, the agent achieved substantially greater efficiency by intentionally favoring brute-force implementations that performed well on the tested input sizes. Although these approaches are theoretically suboptimal, they incurred lower overhead and, in some cases, led to energy reductions exceeding 50%. These gains reflect input-aware simplifications rather than algorithmic superiority.

These findings highlight both the potential and the contextual sensitivity of LLM-generated code. While the agent is capable of producing highly efficient code, its optimizations are often tuned to the specific input characteristics present during measurement. This underscores the importance of careful benchmarking design when evaluating LLM-driven code generation under real-world conditions.

[4] All programs typically execute in under one second. Resulting energy values fall in the millijoule to low-joule range. Task-level absolute measurements and LLM-generated programs are available in the project repository.

5 Discussion

Our results show that EnerAgent can autonomously produce functionally correct and energy-efficient C++ code. Even within a limited benchmark, several insights emerge.

First, modern LLMs like GPT-4.1 achieve high success rates on competitive programming tasks, with over 70% correct on the first try. Many remaining failures are minor and often fixed with one or two repair attempts, highlighting the advantage of iterative, feedback-driven approaches over single-attempt generation.

Second, the optimization loop yields tangible energy gains. Over half of functionally correct solutions were improved, with average savings over 20%. While some reductions fall within measurement noise, manual inspection confirmed they often reflect subtle, but meaningful code changes.

Third, agent-generated solutions frequently match or surpass human-written code in energy usage. In some cases, the agent exploited specific test conditions (e.g. small inputs) to apply simple, low-overhead solutions, such as brute-force approaches that outperformed more general human solutions. While such behavior raises concerns about generalization beyond test conditions, it illustrates the agent's ability to adapt to specific runtime contexts.

EnerAgent demonstrates how LLM agents can serve as practical tools for sustainability-focused software development.

References

[CC25] Chase, H.; Contributors, L.: LangGraph: A low-level orchestration framework for controllable LLM agents, version 0.4.8, Python package, retrieved June 8 2025, 2025, https://github.com/langchain-ai/langgraph.

[CRO24] Cappendijk, T.; de Reus, P.; Oprescu, A.: Generating Energy-efficient code with LLMs. arXiv preprint arXiv:2411.10599, 2024.

[CSL+24] Courty, B.; Schmidt, V.; Luccioni, S., et al.: mlco2/codecarbon: v2.4.1, version v2.4.1, 2024, https://doi.org/10.5281/zenodo.11171501.

[Hu24] Huang, D. et al.: Effilearner: Enhancing efficiency of generated code via self-optimization. Advances in Neural Information Processing Systems 37, pp. 84482–84522, 2024.

[Ji24] Jiang, J. et al.: A survey on large language models for code generation. arXiv preprint arXiv:2406.00515, 2024.

[Ma20] Mann, B. et al.: Language models are few-shot learners. arXiv preprint arXiv:2005.14165 1, p. 3, 2020.

[Ni24] Niu, C. et al.: On evaluating the efficiency of source code generated by llms. In: Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering. Pp. 103–107, 2024.

[Pe24] Peeperkorn, M. et al.: Is temperature the creativity parameter of large language models? arXiv preprint arXiv:2405.00492, 2024.

[Re24] Renze, M.: The effect of sampling temperature on problem solving in large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2024. Pp. 7346–7356, 2024.

[SWC25] Solovyeva, L.; Weidmann, S.; Castor, F.: AI-Powered, But Power-Hungry? Energy Efficiency of LLM-Generated Code. arXiv preprint arXiv:2502.02412, 2025.

[SYL24] Shi, J.; Yang, Z.; Lo, D.: Efficient and green large language models for software engineering: Vision and the road ahead. ACM Transactions on Software Engineering and Methodology, 2024.

[We22] Wei, J. et al.: Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems 35, pp. 24824–24837, 2022.

[Ya23] Yao, S. et al.: React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv. org/abs/2210.03629, 2023.

Appendix: LLM Prompts

LLM Single-Attempt
You are an expert competitive-programming C++ developer. Write clean, efficient, code that follows the template exactly. Only produce C++ code — no commentary.

Reflection Fix Prompt
A programmer tried to solve a problem using C++ code, but the code failed to compile or failed functional tests. In the following you will be presented with a description of the problem, the incorrect code, and the compiler or runtime error logs. Your task: Find the errors in the code and fix them so the code compiles and passes all tests. Keep the same public interface and class structure. Do not add a main() function.

Reflection Optimize Prompt
A programmer solved a problem using C++ code. The code is functionally correct, but may be inefficient. In the following you will be presented with a description of the problem and the code. Your task: Optimize the code to make it as runtime and memory efficient as possible. Make sure the code remains functionally correct and keeps the same public interface and class structure. Do not add a main() function. Other than that you are free to change the code in any way you think increases efficiency. Think step by step how to make the code more efficient and then write the new improved code. If no improvements are possible, you may output `NO_CHANGE`.